Project Workspace & Catalog Architecture (Hierarchy & Ontology)

Active project workspace

We maintain a strict physical separation between the **Codebase Repository** (`IndyRAG/`) and the **Active Project Workspace** (`<project_name>/`). All data, corpus, database, and crop artifacts reside inside the isolated project workspace:

- **`corpus/<machine>/`** — raw source-of-truth PDFs partitioned by machine subfolder (one folder per machine; never modified by the pipeline) under `PROJECT_ROOT`.
- **`library/`** — single shared ordered view over all machines; flat YAML and sqlite databases under `PROJECT_ROOT`.
- **`crops/`** — rendered crop PNGs, plus the centralized `_census` subdivision checklists, `_grid` overlays, and `_scratch` agent renders under `PROJECT_ROOT`.
- **`logs/`** — centralized execution, verification, and discovery log files under `PROJECT_ROOT`.

```
<project_name>/
├── .rag_project                          empty marker file denoting a valid RAG project directory
├── corpus/                               raw PDFs grouped by machine subfolder
│   ├── 3048/                             machine 3048's PDFs
│   └── 3047/                             machine 3047's PDFs
├── library/                              shared ordered view across machines
│   ├── layers.yaml                       layer ontology (immutable once set)
│   ├── regions.yaml                      region proposals + captions
│   ├── violations.yaml                   transient verifier failures
│   ├── chroma/                           persistent vector store
│   ├── fts.sqlite                        keyword index
│   ├── queries.rtf                       batch query input questions
│   ├── responses.txt                     batch query output answers
│   ├── layer_distribution.svg            layer classification SVG distribution plot
│   └── inventory/                        bridging catalogs and files
│       ├── documents.yaml                inventoried source document metadata
```

```
│       ├── assets.yaml                          HITL-approved assets, Uniform Resource Names (URNs), and reported
source types
│       ├── machine_catalog_3047.json            exported catalog for machine 3047
│       └── machine_catalog_3048.json            exported catalog for machine 3048
├── crops/
│   ├── *.png                                    macro-level and detailed visual crop PNG assets
│   ├── drafts.yaml                              persistent drafts queue and Write-Ahead Logging (WAL) log
│   ├── _census/                                 subdivision checklists and progress
│   ├── _grid/                                   grid overviews
│   └── _scratch/                                in-flight agent renders
└── logs/
    ├── .init.ok                                 environment audit and workspace settlement success marker file
    ├── .index_build.ok                          search index build completion success marker file
    └── *.log / *.jsonl                          various execution, verification, and discovery log files
```

Layer ontology

Layers are **user-configurable** and **immutable once set**.

- Defined interactively during the initial ontology setup.
- Stored in `library/layers.yaml`.
- Changing them later requires resetting all derived artifacts.

Field reference

Field	Required	Purpose
`name`	yes	Lowercase identifier; used as the `layer` value on documents and assets.
`prefix`	yes	Uppercase, 2–5 chars. Kept on the layer definition; not embedded in URNs.
`description`	yes	One-liner; injected into the layer-assignment prompt to anchor the per-asset choice.

Field	Required	Purpose
`keywords`	no	Kept for forward compatibility; not consumed by code.

Layer assignment happens at two levels

- **Document level** — a human approves a layer per PDF at HITL Gate 2.
- **Asset level** — the vision model picks one layer per asset from the crop image, caption, vision prompt, and the ontology; the human can override at HITL Gate 4.

Energy vs. Signals disambiguation

The discriminator is **flow type**, not medium:

- A device that **carries power** → `energy` (power flow).
- A device that **carries information** → `signals` (information flow).
- A device with both functions gets two assets, one per layer, anchored to the same region.

Work example — `library/layers.yaml`

The example below is illustrative. Each project picks its own names, prefixes, descriptions, and keywords. The 6-layer set shown here is the canonical ontology the rest of the docs use for examples.

```
layers:
- name: geometrical
  prefix: GEOM
  description: "Machine spatial geometry, dimensions, tolerances, clearance boundaries, and 3D kinemantical
coordinates"
  keywords: [geometry, dimension, clearance, boundary, tolerance, angle, coordination, axis, kinematic, positioning]

- name: hydraulic
```

```
  prefix: HYDR
  description: "Fluid power circuits — hydraulic and pneumatic schematics, valve logic"
  keywords: [hydraulic, pneumatic, valve, cylinder, pump, pressure, flow, filter]

- name: energy
  prefix: ENRG
  description: "Power supply and distribution — electrical, pneumatic, gas supply connection points"
  keywords: [voltage, power, supply, electrical, energy, connection, phase, transformer]

- name: signals
  prefix: SIG
  description: "Information flow — sensors, control signals, HMI displays, alarm tables"
  keywords: [sensor, signal, control, plc, hmi, display, alarm, indicator]

- name: operational
  prefix: STATE
  description: "Operating procedures, system installation, maintenance, troubleshooting — what the operator or
installer does"
  keywords: [installation, maintenance, operation, procedure, troubleshooting, safety, cleaning, lubrication]

- name: reference
  prefix: REF
  description: "Cross-cutting specifications, component bills of materials (BOM), parts catalogs, and general
standards"
  keywords: [specification, requirement, bom, assembly, parts, catalog, listing, foundation]
```

Unique identifier (URN) system

Format

```
asset://<machine>/<doc_id>/IMG-<label>
```

- `<machine>` — the corpus subfolder name (e.g. `3048`). Each machine lives in its own subfolder; the folder name flows through to every document, asset, and URN minted from that folder.
- `<doc_id>` — the source document's identifier (filename-derived, with underscores hyphenated).
- `IMG-` — fixed kind prefix on every minted URN.
- `<label>` — slug of the region's human-readable label.

Properties

- The **layer is not encoded in the URN**. URNs are provenance identifiers, not classification keys.

Filesystem slug

A deterministic mapping turns a URN into a safe filename by replacing non-alphanumeric characters with underscores.

Work example:

```
asset://3048/3048-CAD-159825-ZUSAMMENBAU/IMG-front-elevation
→ asset__3048_3048_CAD_159825_ZUSAMMENBAU_IMG_front_elevation
```

Indexing

Five indexes are built in one pass from the inventory and the corpus.

Index	Backend	Unit	Embedder
`text_chunks`	ChromaDB + FTS5	one record per (page, section) group of merged blocks	`bge-m3`
`captions`	ChromaDB	one record per asset (caption + vision prompt combined)	`bge-m3`
`images`	ChromaDB	one record per crop PNG	`SigLIP`

Index	Backend	Unit	Embedder
`id_search`	FTS5	one record per asset (display name + caption, deduped)	—
`layer_index`	ChromaDB	one record per layer (descriptor: `name + description + keywords`)	`bge-m3`

Design choices

- **Text-rich documents only.** `text_chunks` skips CAD drawings; their content is captured by the captions and images channels.
- **Prose only.** Table-cell shapes and numeric-heavy blocks are dropped from the text search index.
- **Cross-language retrieval.** Non-English page blocks are translated once at build time. The translation is **section-aware**: blocks are sent with their enclosing section heading to ensure technical terminology resolves correctly.
- **Section-aware chunking.** Blocks on a page are merged by their section heading. Lists and procedures stay together as coherent semantic units.
- **No layer prefix in FTS5.** Layer routing lives entirely on the vector side via `layer_index`.
- **Approval required.** Every document and asset must be HITL-approved before indexing.

Retrieval

A query fans out across all channels. Vector hits are re-scored against layer affinity, and Reciprocal Rank Fusion picks the top-K.

Routing weights

Class	Trigger	Channel weights
`LOOKUP`	query contains an asset-ID-like token	text 0.45, captions 0.25, id 0.20, image 0.10
`SEMANTIC`	everything else	image 0.25, text 0.40, captions 0.30, id 0.05
`visual-dominant`	user-requested for visual queries	image 0.60, text 0.05, captions 0.30, id 0.05

Layer affinity re-scoring

Before fusion, vector hits are re-scored using: `score' = α · sim_layer + (1 − α) · sim_passage` where `sim_layer` is the cosine similarity between the query and the hit's layer descriptor.

Synthesis

The synthesizer converts retrieved hits into a cited answer string:

- **Prose citations:** `[doc_id, p.N, hash:<short>]`
- **Asset citations:** `[URN, doc_id, p.N, hash:<short>]`
- **Conflict detection:** Disagreements between sources are surfaced in the answer and recorded in the conflict log.

Operational guarantees

Structural refusals

- URN minting is refused while any document is unapproved.
- Index building is refused unless every document and asset is approved.

Exit codes

Code	Meaning
0	Success
1	General error
2	Sanity check failed
3	Gemini API preflight failed (network or auth)

Code	Meaning
4	Document layer unverified
5	Environment audit incomplete
6	Inventory parse error
7	Index build blocked

Data integrity rules

- **Atomic writes** for all YAML and JSON artifacts.
- **Scratch staging** for renders; promotion only on explicit commit.
- **Pixel cap** on all renders (40 megapixels).
- **Vision-call adaptation.** Oversize crops are downscaled in memory before being sent to the model.

Developer Integration, Version Control, and Automated Hooks

The codebase includes robust integration policies to ensure long-term stability and coordinate multiple concurrent developers:

- **Strict Git Source Isolation:** All raw corpus PDFs, ChromaDB databases, SQLite full-text tables, and JSON-Lines log files are strictly gitignored, keeping our code repository lightweight and securely isolated from project-specific data.
- **Automated Syntax Control Hook:** To prevent syntax drifts or parameter typos from entering our codebase, we enforce a strict multi-tier pre-commit hook. Whenever a developer modifies more than 10 lines of Python code, the hook automatically triggers three sequential passes:
  - *Pass 1:* Grammatical syntax and compile verification.
  - *Pass 2:* Static AST analysis checking call signatures and schema attribute types.
  - *Pass 3:* Comprehensive execution of our complete 207 offline functional test cases, passing 100% green before any code can be committed.

- **Continuous Un-Mocked Live Pipeline Testing:** Visual intelligence and integration endpoints are strictly decoupled to prevent latency during standard local development. When explicitly invoked, the framework executes a continuous, un-mocked visual pipeline test natively against the Gemini API (validating discovery, simulated quality failures, and bounding-box corrections in sequential continuity).

`library/inventory/` — purpose and contents

The inventory is the **bridge between raw bytes and the URN graph**. It answers four questions a single PDF cannot answer on its own:

1. What layer does this document belong to?
2. What real-world assets does it describe? (the URN list)
3. Where in the document does each asset appear? (region, page, bbox, crop)
4. Has the binding been verified, and against which file hash?

It is the only place where layer assignment, URN identity, and region linkage live. Rendering, hashing, and indexing read from it — never the other way around.

On-disk layout

The inventory lives in a directory with two YAML files, mirroring the natural mutation boundary in the pipeline (documents and assets are populated by independent stages):

- **`library/inventory/documents.yaml`** — carries `schema_version`, `layers_hash` (records the ontology hash at seed time so any tampering is detected), and `documents:` (one entry per PDF in `corpus/`).
- **`library/inventory/assets.yaml`** — carries `assets:` only (one entry per HITL-approved region with a minted URN).

The loader merges both files into one validated inventory object — callers see no split. `assets.yaml` is absent in the valid mid-pipeline state between document ingest and URN minting.

Document fields

Field	Source
`doc_id`	derived from filename (uppercased, sanitized, ≤64 chars)
`path`	repo-relative path to the PDF
`sha256`	full hash of the source PDF; first 12 hex chars appear in every citation
`title`	PDF metadata title; falls back to `doc_id`
`language_dominant`, `languages_by_page`	detected per page; **never derived from the filename**
`pages`, `format_class_dominant`	page count and dominant format (`A4` – `A0`, `oversized`)
`layer`	set at HITL Gate 2
`toc`	list of detected section headings (page, block, text, level); built automatically at seed time for text-rich documents only — CAD-shaped documents always have an empty TOC
`needs_verification`, `HITL_approval`	the two-flag lifecycle

Asset fields

Field	Source
`urn`	minted from the approved region
`layer`	starts as `unknown`; filled by the layer-assigner; finalized at HITL Gate 4
`type`	uniform constant `"asset"` for every minted asset; reserved for future differentiation

Field	Source
`display_name`	the region's human-readable label
`doc_id` , `page`	copied from the region for direct citation
`region_id` , `crop_file`	traceability back to the region and its PNG
`caption` , `vision_prompt`	copied from the region for downstream synthesis
`source_type`	copied from the parent document metadata
`needs_verification`	true if automated step run, false if verification passed
`HITL_approval`	true if approved by human at Review Gates

Appendices

Conceptual layer distribution examples

For clarity, the following examples illustrate how specific visual assets are dynamically grouped under the custom layers of our technical ontology:

- **`geometrical` Layer:** Isolates mechanical structural coordinates, isometric drawing views, spatial clearances, geometric boundaries, and 3D relative positioning frames.
- **`reference` Layer:** Combines component specifications, suppliers indices, parts listings, Bills of Materials (BOM) tables, mounting coordinates catalogs, and spare components registries.

Work example — URNs from a two-machine corpus

```
asset://3048/3048-CAD-159825-ZUSAMMENBAU/IMG-isometrics-balloons     (asset.layer = geometrical)
asset://3048/3048-CAD-159825-ZUSAMMENBAU/IMG-bom-table               (asset.layer = reference)
```

```
asset://3048/3048-CAD-162844-AUFSTELLPLAN/IMG-electrical-supply      (asset.layer = energy)
asset://3048/3048-BETRIEBSANLEITUNG-YUKONDAK-SI/IMG-hmi-screen       (asset.layer = signals)
asset://3047/3047-CAD-159888-NACRT-POSTAVITVE/IMG-bom-table          (asset.layer = reference)
asset://3047/3047-BETRIEBSANLEITUNG-DAK-3BL-VTR-SI/IMG-hmi-screen    (asset.layer = signals)
```

The first segment after `asset://` distinguishes machines. Same-label assets on different machines never collide because the machine prefix is part of the doc ID. Note: underscores in the `doc_id` are hyphenated inside the URN's middle segment for readability, even though the YAML `doc_id` field keeps the underscores.

Inventory serialization sample

`library/inventory/documents.yaml`

```yaml
schema_version: "1.0"
layers_hash: 6df395d038762f7b...

documents:
  - doc_id: 3048_BETRIEBSANLEITUNG_YUKONDAK_SI
    machine: "3048"
    path: corpus/3048/3048-Betriebsanleitung-YukonDAK_SI.pdf
    layer: operational
    needs_verification: false
    HITL_approval: true
    sha256: aac6306152f5...
    title: "Betriebsanleitung YukonDAK 3048"
    language_dominant: sl                       # detected, not derived from filename
    languages_by_page: [sl, sl, sl, ...]
    pages: 158
    format_class_dominant: A4

  - doc_id: 3048_CAD_159825_ZUSAMMENBAU
    machine: "3048"
    path: corpus/3048/CAD-159825_Zusammenbau.pdf
    layer: geometrical
```

```
    needs_verification: false
    HITL_approval: true
    sha256: aac6306152f5...
    title: "Zusammenbauzeichnung 3048"
    language_dominant: de
    languages_by_page: [de]
    pages: 1
    format_class_dominant: A0
```

library/inventory/assets.yaml

```
assets:
  - urn: asset://3048/3048-CAD-159825-ZUSAMMENBAU/IMG-isometrics-balloons
    machine: "3048"
    layer: geometrical
    type: assembly_position
    display_name: "isometrics_balloons"
    doc_id: 3048_CAD_159825_ZUSAMMENBAU
    page: 1
    region_id: 3048_CAD_159825_ZUSAMMENBAU_p01_isometrics_balloons
    crop_file: crops/3048_CAD_159825_ZUSAMMENBAU_p01_isometrics_balloons.png
    caption: "Isometric rendering with 23 balloon callouts"
    vision_prompt: "The crop shows..."
    source_type: "A0 Zusammenbau 3048"
    needs_verification: false                  # automated layer assignment ran
    HITL_approval: true                        # human approved at Gate 4

  - urn: asset://3047/3047-CAD-159888-NACRT-POSTAVITVE/IMG-bom-table
    machine: "3047"
    layer: reference
    type: asset
    display_name: "bom_table"
    doc_id: 3047_CAD_159888_NACRT_POSTAVITVE
    page: 1
    region_id: 3047_CAD_159888_NACRT_POSTAVITVE_p01_bom_table
```

```
    crop_file: crops/3047_CAD_159888_NACRT_POSTAVITVE_p01_bom_table.png
    caption: "Bill of materials"
    vision_prompt: "Standard tabbed table with components..."
    source_type: "A1 Nacrt Postavitve"
    needs_verification: false
    HITL_approval: true
```